

# PYTHON INTERVIEW REVISION SHEET (COMPRESSED)

## 1. PYTHON BASICS

### Q1. What is Python and why is it widely used?

**Answer:** Python is a high-level, interpreted, dynamically typed language known for readability and a vast ecosystem (NumPy, FastAPI, TensorFlow). It enables rapid development across backend, data science, and AI systems with strong community and library support.

### Q2. What is dynamic typing and how is Python still strongly typed?

**Answer:** Python determines variable types at runtime (dynamic typing) but enforces strict type rules (strong typing), preventing invalid operations like "1" + 2. This balance allows flexibility while avoiding silent type coercion errors.

### Q3. Explain mutable vs immutable objects with memory implications.

**Answer:** Mutable objects (list, dict) can change in-place, while immutable ones (int, str, tuple) create new objects on modification. This affects memory usage, references, and side effects in shared data structures.

### Q4. How does Python manage memory?

**Answer:** Python uses reference counting for immediate cleanup and garbage collection for cyclic references, all within a private heap. This ensures automatic memory management but requires awareness to avoid leaks in complex objects.

### Q5. What is the difference between list, tuple, set, and dict?

**Answer:** Lists are ordered and mutable, tuples are immutable, sets store unique unordered elements, and dicts store key-value pairs with O(1) average lookup. Choosing the right structure impacts performance and design.

## 2. DATA STRUCTURES & ALGORITHMS

### Q6. Two Sum Problem

**Answer:** Use a hash map to store complements and achieve  $O(n)$  time and  $O(n)$  space complexity instead of  $O(n^2)$  brute force. Handles negatives and duplicates efficiently with constant-time lookup.

### Q7. What is the difference between array (list) and linked list?

**Answer:** Python lists are dynamic arrays with  $O(1)$  access but  $O(n)$  insertion, while linked lists offer  $O(1)$  insertion but  $O(n)$  access. Lists are cache-friendly; linked lists are pointer-based and rarely used in Python.

### Q8. What is time complexity and why does it matter?

**Answer:** Time complexity measures algorithm scalability using Big-O notation ( $O(1)$ ,  $O(n)$ ,  $O(\log n)$ ). It determines performance under large inputs and is critical for writing efficient production code.

### Q9. Explain hash maps and collisions

**Answer:** Hash maps use hash functions to map keys to indices with  $O(1)$  average lookup, but collisions occur when keys share indices. Python handles collisions internally using optimized probing techniques.

### Q10. What is a sliding window technique?

**Answer:** Sliding window optimizes subarray/substring problems by maintaining a dynamic window, reducing complexity from  $O(n^2)$  to  $O(n)$ . Common in streaming and string problems.

### Q11. Longest substring without repeating characters

**Answer:** Use sliding window with a set to maintain uniqueness, achieving  $O(n)$  time and space. Adjust left pointer when duplicates appear to maintain valid window.

**Q12. What is two pointers technique?**

**Answer:** Two pointers traverse data from different ends or speeds, reducing nested loops to  $O(n)$ . Common in sorted arrays and palindrome problems.

**Q13. Find pair with target sum (sorted array)**

**Answer:** Use two pointers (left/right) to achieve  $O(n)$  time and  $O(1)$  space, adjusting pointers based on sum comparison. More efficient than hash maps for sorted inputs.

**Q14. What is recursion and when should you avoid it?**

**Answer:** Recursion solves problems via self-calls with base cases but risks stack overflow and higher space complexity ( $O(n)$ ). Avoid in deep recursion; prefer iterative or DP approaches.

**Q15. Factorial using recursion**

**Answer:** Recursive solution has  $O(n)$  time and  $O(n)$  stack space; iterative avoids recursion overhead. Must handle edge cases like  $n=0$  and negative input.

**Q16. What is dynamic programming?**

**Answer:** DP stores subproblem results (memoization/tabulation) to avoid recomputation, reducing exponential complexity to polynomial. Used in optimization and overlapping subproblems.

**Q17. Fibonacci using DP**

**Answer:** Bottom-up DP achieves  $O(n)$  time and  $O(1)/O(n)$  space, avoiding exponential recursion. Preferred in interviews for efficiency.

**Q18. What is a stack and where is it used?**

**Answer:** Stack is LIFO structure used in recursion, expression evaluation, and backtracking. Operations (push/pop) are  $O(1)$ .

**Q19. Valid parentheses problem**

**Answer:** Use stack to match opening/closing brackets with  $O(n)$  time and  $O(n)$  space. Ensures proper nesting using mapping of pairs.

**Q20. What is a queue?**

**Answer:** Queue is FIFO structure used in BFS and scheduling systems. Implemented via `collections.deque` for  $O(1)$  operations.

**Q21. BFS vs DFS**

**Answer:** BFS explores level-wise using queue (shortest path), DFS explores depth-first using stack/recursion. Choice depends on problem requirements.

**Q22. What is binary search?**

**Answer:** Binary search reduces search space by half each step, achieving  $O(\log n)$  time on sorted arrays. Requires sorted input and careful boundary handling.

**Q23. Binary search implementation**

**Answer:** Iterative binary search uses two pointers with  $O(\log n)$  time and  $O(1)$  space, avoiding recursion overhead. Must handle edge cases like duplicates.

**Q24. What is a heap?**

**Answer:** Heap is a binary tree used for priority queues with  $O(\log n)$  insertion and extraction. Implemented using `heapq` in Python.

**Q25. Top K elements problem**

**Answer:** Use heap (min/max) to achieve  $O(n \log k)$  complexity instead of sorting  $O(n \log n)$ . Efficient for large datasets.

### 3. OOP

#### **Q26. What is Object-Oriented Programming and why is it important?**

**Answer:** OOP organizes code using classes and objects to improve modularity, reusability, and scalability. It models real-world systems and supports maintainable architecture.

#### **Q27. What is a class and an object?**

**Answer:** A class is a blueprint defining attributes/methods, while an object is its instance. Enables structured design and encapsulation of logic.

#### **Q28. What is encapsulation?**

**Answer:** Encapsulation hides internal data using protected/private attributes, exposing controlled interfaces. Improves data safety and abstraction.

#### **Q29. What is inheritance?**

**Answer:** Inheritance allows child classes to reuse parent functionality, promoting code reuse and extensibility. Supports hierarchical design.

#### **Q30. What is polymorphism?**

**Answer:** Polymorphism allows same interface to behave differently across objects. Enables flexible and interchangeable components.

#### **Q31. Method overriding vs overloading**

**Answer:** Overriding modifies parent methods in child classes, while Python simulates overloading via default arguments. True overloading is not natively supported.

**Q32. What is abstraction?**

**Answer:** Abstraction hides complexity by exposing only essential features. Helps focus on “what” rather than “how”.

**Q33. What are dunder (magic) methods?**

**Answer:** Special methods like `__init__`, `__str__` enable operator overloading and built-in behavior customization. Used for object representation and control.

**Q34. What is composition vs inheritance?**

**Answer:** Composition uses objects within objects, offering flexibility, while inheritance extends behavior. Composition is preferred for loose coupling.

 **4. PYTHON INTERNALS & CONCURRENCY****Q35. What is GIL (Global Interpreter Lock)?**

**Answer:** GIL ensures only one thread executes Python bytecode at a time in CPython, limiting CPU-bound multithreading. Multiprocessing bypasses this limitation.

**Q36. How does Python execute code internally?**

**Answer:** Python compiles `.py` code into bytecode (`.pyc`), then executes it using a virtual machine. This hybrid model balances flexibility and performance.

**Q37. What is reference counting?**

**Answer:** Python tracks object references and frees memory when count reaches zero. It is fast but cannot handle cyclic references alone.

**Q38. What is garbage collection?**

**Answer:** GC handles cyclic references using periodic cleanup, complementing reference counting. Prevents memory leaks in complex object graphs.

**Q39. What are Python namespaces?**

**Answer:** Namespaces map variable names to objects, organized as local, global, and built-in scopes. Prevents naming conflicts.

**Q40. What is LEGB rule?**

**Answer:** Variable lookup follows Local → Enclosing → Global → Built-in order. Determines scope resolution during execution.

**Q41. What is multithreading?**

**Answer:** Multithreading allows concurrent execution but is limited by GIL for CPU-bound tasks. Best suited for I/O-bound operations.

**Q42. What is multiprocessing?**

**Answer:** Multiprocessing runs separate processes with independent memory, enabling true parallelism. Used for CPU-intensive tasks.

**Q43. Difference between threading and multiprocessing**

**Answer:** Threads share memory but are GIL-bound; processes use separate memory and achieve parallelism. Trade-off between speed and overhead.

**Q44. What is async programming?**

**Answer:** Async enables non-blocking execution using event loops, improving concurrency for I/O tasks. Reduces thread overhead.

**Q45. What is async/await?**

**Answer:** async/await allows cooperative multitasking by suspending tasks until awaited operations complete. Essential for scalable APIs.

**Q46. When to use async vs threads vs multiprocessing?**

**Answer:** Async for I/O-bound tasks, threads for light concurrency, multiprocessing for CPU-bound tasks. Choosing correctly optimizes performance.

**Q47. What is event loop?**

**Answer:** Event loop manages async task execution, scheduling and switching between coroutines efficiently. Core of async frameworks.

**Q48. What is blocking vs non-blocking code?**

**Answer:** Blocking waits for completion, while non-blocking continues execution. Non-blocking improves responsiveness in concurrent systems.

**Q49. Example async API call**

**Answer:** Async functions use await for I/O operations, enabling efficient concurrency. Reduces latency in API-heavy systems.

**Q50. Why async is important in backend systems?**

**Answer:** Async handles high concurrency with fewer resources, making it ideal for APIs and microservices. Improves throughput and scalability.

 **PYTHON INTERVIEW REVISION SHEET (PART 2)**

 **5. BACKEND DEVELOPMENT**

**Q51. What is a REST API and how does it work?**

**Answer:** REST APIs expose resources via HTTP methods (GET, POST, PUT, DELETE) and are stateless, meaning each request contains all required context. They enable standardized communication between client and server systems.



**Q52. What is FastAPI and why is it popular?**

**Answer:** FastAPI is an async-first Python framework using type hints and ASGI, offering high performance and automatic validation/docs. It is widely used for scalable APIs and ML model serving.

**Q53. Difference between Flask, Django, and FastAPI**

**Answer:** Flask is lightweight and flexible, Django is full-stack with built-in features, and FastAPI is async and performance-oriented. Choice depends on project scale and concurrency needs.

**Q54. What is ASGI vs WSGI?**

**Answer:** WSGI supports synchronous apps, while ASGI supports asynchronous execution and WebSockets. Modern systems prefer ASGI for high concurrency.

**Q55. How does request lifecycle work in FastAPI?**

**Answer:** Request flows through routing → validation (Pydantic) → business logic → response serialization. Ensures clean input/output handling with type safety.

**Q56. What is dependency injection in FastAPI?**

**Answer:** Dependency injection uses Depends() to manage reusable components like DB connections or auth logic. Improves modularity and testability.

**Q57. What is middleware?**

**Answer:** Middleware intercepts requests/responses globally for logging, authentication, or error handling. It operates before and after route execution.

**Q58. How do you handle authentication in APIs?**

**Answer:** Common methods include JWT, OAuth2, and API keys, ensuring only authorized users access resources. Authentication verifies identity; authorization controls access.

**Q59. What is JWT and how does it work?**

**Answer:** JWT is a stateless token containing payload and signature, sent with each request for authentication. It avoids server-side session storage.

**Q60. What is rate limiting?**

**Answer:** Rate limiting restricts the number of requests per user/time window to prevent abuse and ensure system stability. Implemented via Redis or API gateways.

**Q61. What is caching and why is it important?**

**Answer:** Caching stores frequently accessed data (e.g., Redis) to reduce DB load and latency. Improves performance but requires cache invalidation strategies.

**Q62. What is Redis and how is it used?**

**Answer:** Redis is an in-memory data store used for caching, session management, and queues. Provides ultra-fast data access with  $O(1)$  operations.

 **6. DATABASES**

**Q63. SQL vs NoSQL**

**Answer:** SQL databases are structured and ACID-compliant, while NoSQL offers flexible schemas and horizontal scalability. Choice depends on consistency vs scalability needs.

**Q64. What is indexing?**

**Answer:** Indexing improves query speed by reducing search time from  $O(n)$  to  $O(\log n)$ , but increases write overhead and storage cost.

**Q65. What is normalization?**

**Answer:** Normalization organizes data to reduce redundancy and improve consistency, often through multiple related tables.

**Q66. What is a transaction?**

**Answer:** A transaction ensures ACID properties (Atomicity, Consistency, Isolation, Durability) for reliable database operations.

**Q67. What is ORM?**

**Answer:** ORM maps database tables to Python objects (e.g., SQLAlchemy), simplifying DB interactions and reducing SQL complexity.

**Q68. What are joins?**

**Answer:** Joins combine data from multiple tables based on relationships (INNER, LEFT, RIGHT). Essential for relational queries.

 **7. SCALING & PRODUCTION**

**Q69. What is horizontal vs vertical scaling?**

**Answer:** Vertical scaling increases machine resources, while horizontal scaling adds more machines. Distributed systems prefer horizontal scaling.

**Q70. What is load balancing?**

**Answer:** Load balancers distribute incoming traffic across servers to ensure availability and prevent overload.

**Q71. What is microservices architecture?**

**Answer:** Microservices split applications into independent services communicating via APIs, improving scalability and maintainability.

**Q72. Monolith vs Microservices**

**Answer:** Monolith is a single codebase, easier to start but hard to scale; microservices are distributed, scalable but complex to manage.

**Q73. What is API gateway?**

**Answer:** API gateway acts as a single entry point for routing, authentication, and rate limiting in microservices architecture.

**Q74. What is message queue?**

**Answer:** Message queues (Kafka, RabbitMQ) enable asynchronous communication between services, improving decoupling and scalability.

**Q75. What is idempotency in APIs?**

**Answer:** Idempotent operations produce the same result regardless of repeated requests, ensuring safe retries.

**Q76. How to design scalable API?**

**Answer:** Use stateless design, caching, load balancing, async processing, and proper DB scaling to handle high traffic.

**Q77. What is connection pooling?**

**Answer:** Connection pooling reuses database connections to reduce overhead and improve performance.

**Q78. What is logging?**

**Answer:** Logging records system events for debugging, monitoring, and auditing. Critical for production observability.

**Q79. What is monitoring?**

**Answer:** Monitoring tracks metrics like latency, CPU, memory, and errors to ensure system health.

**Q80. What is CI/CD?**

**Answer:** CI/CD automates building, testing, and deploying applications, enabling faster and reliable releases.

 **PYTHON INTERVIEW REVISION SHEET (PART 3)**

 **8. NUMPY**

**Q81. What is NumPy and why is it faster than Python lists?**

**Answer:** NumPy uses contiguous memory and C-optimized vectorized operations, avoiding Python loops and enabling SIMD execution. This results in significantly faster numerical computation compared to lists.

**Q82. What is a NumPy array (ndarray)?**

**Answer:** ndarray is a fixed-size, homogeneous multi-dimensional array optimized for numerical operations. It supports efficient memory layout and broadcasting.

**Q83. What is vectorization?**

**Answer:** Vectorization replaces loops with array operations, improving speed and readability. It leverages low-level optimizations to perform bulk computations efficiently.

**Q84. What is broadcasting in NumPy?**

**Answer:** Broadcasting allows operations on arrays of different shapes by automatically expanding dimensions. It avoids copying data, making computations memory-efficient.

**Q85. Difference between NumPy array and Python list**

**Answer:** NumPy arrays are homogeneous, faster, and memory-efficient, while Python lists are flexible but slower and less efficient. NumPy is preferred for numerical workloads.

**Q86. What is slicing in NumPy?**

**Answer:** NumPy slicing returns views instead of copies, meaning changes affect the original array. This improves performance but can cause unintended side effects.

**Q87. What is reshaping?**

**Answer:** Reshaping changes array dimensions without altering data, enabling flexible data transformations. It requires compatible total element count.

**Q88. What is axis in NumPy?**

**Answer:** Axis defines direction of operations (axis=0 for columns, axis=1 for rows). It controls how aggregation functions are applied.

**9. PANDAS****Q89. What is Pandas?**

**Answer:** Pandas is a data analysis library providing DataFrame and Series structures for handling structured data efficiently. It supports powerful data manipulation and cleaning operations.

**Q90. What is a DataFrame?**

**Answer:** A DataFrame is a 2D labeled data structure with rows and columns, similar to a table. It supports heterogeneous data types and indexing.

**Q91. How do you handle missing values?**

**Answer:** Missing values are handled using `dropna()` or `fillna()` depending on context. Imputation strategies depend on data distribution and use case.

**Q92. What is groupby?**

**Answer:** groupby splits data into groups and applies aggregation functions like sum, mean, or count. It is essential for data summarization.

**Q93. What is merge vs join?**

**Answer:** Merge performs SQL-like joins on columns, while join operates on indices. Both combine datasets but differ in usage context.

**Q94. What is apply vs map?**

**Answer:** map applies element-wise transformations on Series, while apply operates row/column-wise on DataFrames. apply is more flexible but slower.

**Q95. How to optimize Pandas performance?**

**Answer:** Use vectorization, avoid loops/apply, leverage categoricals, and minimize copies. Efficient operations significantly reduce execution time.

**Q96. What are common Pandas pitfalls?**

**Answer:** Issues include chained indexing, SettingWithCopyWarning, and inefficient loops. Understanding views vs copies is critical for correctness.

 **10. STATISTICS****Q97. What is mean, median, mode?**

**Answer:** Mean is average, median is middle value, and mode is most frequent value. Each represents central tendency differently depending on distribution.

**Q98. What is standard deviation?**

**Answer:** Standard deviation measures data spread around the mean. Higher values indicate more variability.

**Q99. What is variance?**

**Answer:** Variance is the square of standard deviation, representing dispersion of data points. It penalizes larger deviations more.

**Q100. What is normal distribution?**

**Answer:** Normal distribution is a symmetric bell-shaped curve where mean = median = mode. Many real-world phenomena follow this distribution.

**Q101. What is bias vs variance?**

**Answer:** Bias causes underfitting due to oversimplification, while variance causes overfitting due to sensitivity to data. Balancing both is key.

**Q102. What is correlation?**

**Answer:** Correlation measures relationship strength between variables, ranging from -1 to 1. It does not imply causation.

**Q103. What is p-value?**

**Answer:** P-value indicates probability of observing results under null hypothesis. Low p-values suggest statistical significance.

 **11. DATA CODING**

**Q104. Find missing values in dataset**

**Answer:** Use `df.isnull().sum()` to identify missing values column-wise. Helps in preprocessing and data cleaning.

**Q105. Remove duplicates**

**Answer:** `df.drop_duplicates()` removes repeated rows, ensuring data consistency. Useful in preprocessing pipelines.



**Q106. Normalize data**

**Answer:**  $(df - df.min()) / (df.max() - df.min())$  scales values to 0–1 range. Important for ML models sensitive to scale.

**Q107. Group and aggregate**

**Answer:** `df.groupby("col")["val"].mean()` groups data and computes aggregates efficiently. Used in analytics and reporting.

**Q108. Filter data**

**Answer:** `df[df["col"] > 10]` filters rows based on conditions. Enables quick data slicing.

 **12. MACHINE LEARNING****Q109. What is Machine Learning?**

**Answer:** ML enables systems to learn patterns from data and make predictions without explicit programming. It focuses on generalization from training data.

**Q110. Types of Machine Learning**

**Answer:** Supervised uses labeled data, unsupervised finds patterns without labels, and reinforcement learns via rewards. Each suits different problem types.

**Q111. What is overfitting and underfitting?**

**Answer:** Overfitting memorizes training data, while underfitting fails to capture patterns. Both reduce model generalization.

**Q112. What is bias vs variance?**

**Answer:** Bias causes systematic error, variance causes sensitivity to data changes. Trade-off determines model performance.

**Q113. What is train/test split?**

**Answer:** Data is split into training and testing sets to evaluate model performance on unseen data. Prevents data leakage.

**Q114. What is cross-validation?**

**Answer:** Cross-validation splits data multiple times to ensure model stability and robust evaluation. Reduces variance in performance estimates.

**Q115. What is precision, recall, F1-score?**

**Answer:** Precision measures accuracy of positives, recall measures coverage, and F1 balances both. Used in classification evaluation.

**Q116. What is confusion matrix?**

**Answer:** Matrix showing TP, FP, TN, FN for classification results. Helps analyze model performance in detail.

**Q117. What is ROC-AUC?**

**Answer:** ROC-AUC measures model performance across thresholds, indicating classification separability. Higher is better.

**Q118. What is feature engineering?**

**Answer:** Creating meaningful input features improves model performance. It is often more impactful than model choice.

**Q119. What is normalization vs standardization?**

**Answer:** Normalization scales to 0–1, while standardization centers data (mean=0, std=1). Choice depends on model requirements.

**Q120. What is gradient descent?**

**Answer:** Gradient descent minimizes loss by iteratively updating parameters. Core optimization algorithm in ML.

**Q121. What is loss function?**

**Answer:** Loss function quantifies prediction error, guiding model optimization. Different tasks use different loss types.

**Q122. What is regularization?**

**Answer:** Regularization prevents overfitting by penalizing large weights (L1/L2). Improves generalization.

 **13. GENERATIVE AI / LLM**

**Q128. What is a Large Language Model (LLM)?**

**Answer:** LLMs are deep neural networks trained on massive text data to generate and understand language. Examples include GPT and BERT.

**Q129. What is transformer architecture?**

**Answer:** Transformers use attention mechanisms instead of recurrence, enabling parallel processing and better context handling.

**Q130. What is attention mechanism?**

**Answer:** Attention allows models to focus on relevant parts of input dynamically. Improves context understanding.

**Q131. What is fine-tuning?**

**Answer:** Fine-tuning adapts pre-trained models to specific tasks using smaller datasets. Improves performance on domain-specific tasks.

**Q132. What is prompt engineering?**

**Answer:** Prompt engineering designs inputs to control LLM outputs effectively. Critical for guiding responses.

**Q133. What is embedding?**

**Answer:** Embeddings convert text into numerical vectors capturing semantic meaning. Used in similarity search and ML models.

**Q134. What is vector database?**

**Answer:** Vector DB stores embeddings for efficient similarity search. Used in semantic search and RAG systems.

**Q135. What is RAG (Retrieval-Augmented Generation)?**

**Answer:** RAG combines retrieval systems with LLMs to improve accuracy using external knowledge sources.

**Q136. What are hallucinations in LLMs?**

**Answer:** Hallucinations are incorrect but confident outputs generated by LLMs. A key limitation of generative AI.

**Q137. How to reduce hallucinations?**

**Answer:** Use RAG, better prompts, and fine-tuning to ground responses in real data.

**Q138. What is tokenization?**

**Answer:** Tokenization splits text into smaller units (tokens) for model processing. Affects model efficiency and cost.

**Q139. What is temperature in LLM?**

**Answer:** Temperature controls randomness in output; low = deterministic, high = creative.

**Q140. What are real-world LLM applications?**

**Answer:** Chatbots, summarization, search, and code generation are key applications. Widely used in production systems.



## **14. SYSTEM DESIGN**

**Q141. What is System Design and why is it important?**

**Answer:** System design defines architecture for scalable, reliable systems, including components and data flow. Critical for handling large-scale applications.

**Q142. What are scalability and availability?**

**Answer:** Scalability handles increased load, while availability ensures uptime. Both are essential for production systems.

**Q143. What is horizontal vs vertical scaling?**

**Answer:** Vertical scaling increases resources of one machine, while horizontal adds more machines. Distributed systems prefer horizontal scaling.

**Q144. What is load balancing?**

**Answer:** Load balancing distributes traffic across servers to ensure reliability and performance.

**Q145. What is caching in system design?**

**Answer:** Caching stores frequently accessed data (Redis/CDN) to reduce latency and backend load.

**Q146. What is database sharding?**

**Answer:** Sharding splits data across multiple databases to improve scalability and performance.

**Q147. What is replication?**

**Answer:** Replication duplicates data across servers for fault tolerance and availability.

**Q148. What is CAP theorem?**

**Answer:** CAP states systems can guarantee only two of Consistency, Availability, and Partition tolerance in distributed systems.

**Q149. What is eventual consistency?**

**Answer:** System becomes consistent over time rather than instantly. Used in distributed databases.

**Q150. What is rate limiting in system design?**

**Answer:** Limits requests per user to prevent abuse and maintain stability.

**Q151. How would you design a URL shortener?**

**Answer:** Use hashing for short IDs, store mappings in DB, and cache frequently accessed URLs. Ensure scalability and uniqueness.

**Q152. How would you design a scalable API service?**

**Answer:** Use stateless design, load balancing, caching, async processing, and DB scaling to handle high traffic.

**Q153. How to handle millions of requests?**

**Answer:** Use horizontal scaling, caching, CDNs, async processing, and efficient load balancing.

**Q154. How to design a chat system?**

**Answer:** Use WebSockets for real-time communication, message queues for reliability, and scalable storage.

**Q155. What is API versioning?**

**Answer:** API versioning ensures backward compatibility when updating APIs.

**Q156. How would you deploy an ML model?**

**Answer:** Train model, serialize (pickle/joblib), deploy via API (FastAPI), and monitor performance.

**Q157. What is model drift?**

**Answer:** Model drift occurs when performance degrades due to changing data patterns.

**Q158. How to monitor ML systems?**

**Answer:** Track accuracy, latency, and data drift to ensure model reliability in production.

**Q159. What is batch vs real-time inference?**

**Answer:** Batch processes data offline, while real-time provides instant predictions.

**Q160. How would you design an LLM-powered application?**

**Answer:** Combine LLM API, vector DB, prompt layer, and caching for scalable architecture.

**Q161. What is RAG architecture?**

**Answer:** RAG integrates retrieval with generation to improve accuracy using external knowledge.

**Q162. How to scale LLM systems?**

**Answer:** Use caching, batching, prompt optimization, and efficient retrieval systems.

**Q163. What are latency challenges in AI systems?**

**Answer:** Latency arises from model size, inference time, and network delays.

**Q164. What is observability in systems?**

**Answer:** Observability involves monitoring logs, metrics, and traces for system health.

**Q165. What is fault tolerance?**

**Answer:** Fault tolerance ensures systems continue functioning despite failures.

**Q166. What is circuit breaker pattern?**

**Answer:** Circuit breaker prevents repeated failures by stopping calls to failing services.

**Q167. What is retry mechanism?**

**Answer:** Retry mechanism reattempts failed operations safely with backoff strategies.

**Q168. What is idempotency in distributed systems?**

**Answer:** Ensures repeated requests produce the same result without side effects.

**Q169. What is data pipeline?**

**Answer:** Data pipeline moves data from source → processing → storage efficiently.

**Q170. What is ETL?**

**Answer:** ETL stands for Extract, Transform, Load, used in data engineering workflows.